AI Solution Architect Agent

Project Explainer — Logic, Tech Stack & Agent Roles
A fast, mentor-ready walkthrough of what the project does and how it works

In one line: Upload a client requirement (even a messy email or PDF) and the system returns a complete, structured solution blueprint — requirement analysis, architecture, tech stack, effort estimate, risks, and a client-ready proposal — in minutes.

1 · The Agenda — Why This Project Exists

Today, when a client shares a requirement document, presales teams and solution architects spend **hours to days** manually doing the same work: reading the requirement, choosing technologies, estimating effort, drawing architecture, listing risks, and writing a proposal.

The goal: automate that entire presales / solution-design process so a consulting firm (e.g. Algo8) can produce a professional solution blueprint in minutes instead of hours — cutting cost and turnaround time.

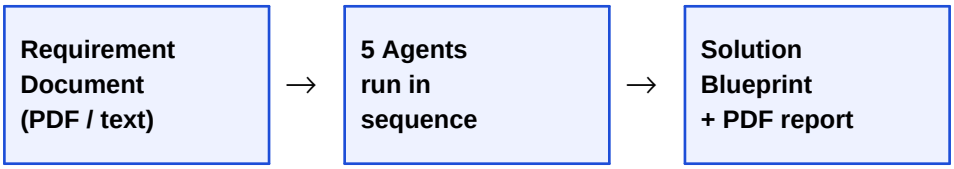
Who uses it: Presales teams, Solution Architects, Delivery Managers, and Technical Consultants.

2 · How It Works — The Core Idea

The system is a **lightweight pipeline of 5 specialised agents** that run one after another. Each agent is an expert at one job and passes its output to the next. There is no complex multi-agent orchestration — just a clean, explainable sequence.

Unstructured in → Structured out. The first agent uses GPT-4o to turn ANY messy text (email, notes, PDF) into structured fields. Every later agent builds on that structure.

The flow:



Behind the scenes, a single **orchestrator** (orchestrator.py) calls each agent in order and assembles one final object called the **SolutionBlueprint**. Both the Streamlit UI and the optional FastAPI backend use this same orchestrator — one source of truth.

3 · The 5 Agents — Who Does What

Each agent lives in its own file under `agents/` and exposes one `run()` function.

#	Agent	Takes In	Produces
1	**Requirement Agent** requirement_agent.py	Raw requirement text (from PDF / email / paste)	Project name, business goal, target users, key features, integrations, constraints
2	**Architecture Agent** architecture_agent.py	The structured requirement	Recommended architecture, full tech stack, and a Mermaid architecture diagram
3	**Estimation Agent** estimation_agent.py	Requirement + architecture	Effort per phase (person-days), total, and the recommended team structure
4	**Risk Agent** risk_agent.py	Requirement + architecture	A risk register: each risk with impact, likelihood, and a mitigation
5	**Proposal Agent** proposal_agent.py	All of the above combined	Client-ready proposal: executive summary, solution, timeline, assumptions, next steps

Key design detail: every agent first tries GPT-4o; if that fails or no API key is set, it falls back to a built-in rule-based generator so the demo *never breaks*. (See section 6: Two Modes.)

4 · Tech Stack — What We Use & Why

Layer	Technology	Why we chose it
Frontend / UI	**Streamlit**	Builds a clean, modern single-page web UI in pure Python — no HTML/JS needed. Perfect for fast demos.
Backend (optional API)	**FastAPI**	Lightweight, fast Python web framework. Exposes the pipeline as REST endpoints for other apps.
AI Framework	**LangChain**	Standard toolkit to talk to LLMs cleanly — manages prompts and model calls.
LLM (the brain)	**OpenAI GPT-4o**	Does the actual understanding: reads messy text and reasons about architecture, effort, and risks.
PDF input	**PyPDF**	Extracts text from uploaded requirement PDFs.
PDF output	**fpdf2**	Generates the downloadable solution-blueprint report.
Data models	**Pydantic**	Defines a typed contract for every agent's output — keeps data clean and predictable.
Diagram	**Mermaid**	Text-based diagrams — the architecture diagram is generated as code and rendered in the browser.

Deliberately kept simple: no authentication, no Docker, no Kubernetes, no cloud setup — so it is easy to run locally and easy to explain.

5 · Code Structure — The Key Files

File / Folder	What it does
`app.py`	The Streamlit UI — upload box, the 9 result sections, and download buttons.
`orchestrator.py`	The 'conductor'. Runs all 5 agents in order and builds the final SolutionBlueprint.
`agents/`	The 5 agent files (+ base.py, which holds the shared LLM connection and demo-mode logic).
`prompts/`	One prompt template per agent — the instructions we give GPT-4o. Easy to read and tune.
`utils/schemas.py`	Pydantic data models — the typed shape of every agent's output.
`utils/pdf_utils.py`	Reads text out of uploaded PDF / TXT / MD files.
`utils/mermaid.py`	Builds the architecture diagram as Mermaid code.
`utils/report.py`	Exports the final blueprint as a downloadable PDF and Markdown.
`backend/main.py`	Optional FastAPI REST API (the UI works without it).
`sample/`	A ready-made sample requirement so you can demo instantly.

---

6 · Two Modes — GPT-4o vs Demo

- **GPT-4o mode** (an OpenAI API key is set in the .env file): real LLM intelligence — handles messy, unstructured input and produces tailored output.
- **Demo mode** (no key): each agent falls back to smart keyword-based rules, so the app still produces a full blueprint completely offline — ideal for a guaranteed workshop demo.

The app shows a banner at the top telling you which mode is active.

---

7 · How To Run It

1) Activate the environment and start the app:

```
cd AI_Architect_Solution
.venv\Scripts\activate
streamlit run app.py
```

2) Open **http://localhost:8501** → paste a requirement (or click *Load sample*) → click **Generate Solution Blueprint** → download the PDF.

8 · 2-Minute Talk Track for Your Mentor

Use this script to explain the project confidently:

- **The problem:** 'Presales teams spend hours turning a client requirement into a proposal. I automated that.'
- **The idea:** 'You upload any requirement — even a messy email — and my system returns a full structured solution blueprint in minutes.'
- **How:** 'It's a pipeline of 5 specialised agents. Each does one job and feeds the next, coordinated by a single orchestrator.'
- **The 5 agents:** 'Requirement → Architecture → Estimation → Risk → Proposal. The first one uses GPT-4o to convert unstructured text into structured data; the rest build on it.'
- **The tech:** 'Streamlit for the UI, FastAPI for the API, LangChain + GPT-4o for the AI, PyPDF to read PDFs, fpdf2 to generate the report, and Pydantic to keep the data clean.'
- **The smart bit:** 'Every agent has a fallback, so the demo works even without an internet connection or API key.'
- **The output:** 'On screen you see requirement analysis, an architecture diagram, the tech stack, effort & team, a risk table, and a proposal — all downloadable as a PDF.'

End of explainer — AI Solution Architect Agent